

**МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ «ЛЭТИ» ИМ.В.И.УЛЬЯНОВА (ЛЕНИНА)»**

**ФАКУЛЬТЕТ КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ И ИНФОРМАТИКИ
КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ**

**Зачётная работа №3
по дисциплине «Алгоритмы и структуры данных»
на тему «Деревья»**

Выполнили студенты группы 3312

Лебедев И.А.

Шарапов И.Д.

Принял старший преподаватель Колинько П.Г.

Санкт-Петербург
2024

Содержание

Цель работы	3
Описание задания	3
Обоснование выбора способа представления деревьев в памяти ЭВМ	3
Результаты прогона программы с генерацией случайного дерева	4
Оценки временной сложности для каждой функции	4
Выводы	5
Список использованной литературы.....	5
Приложение	6

Цель работы

Исследование алгоритмов для работы с двоичным деревом.

Описание задания

Реализовать программу для работы с двоичным деревом с разметкой узлов в обратном порядке. Программа должна выполнять обход дерева в глубину (DFS) и вычислять высоту левого поддерева корня.

Обоснование выбора способа представления деревьев в памяти ЭВМ

Для представления дерева в памяти была выбрана структура, основанная на динамических объектах с указателями на дочерние узлы. Каждый узел дерева представлен классом *Node*, который содержит метку узла (*char*) и указатели на левого и правого потомков (*Node**). Такой подход обеспечивает необходимую гибкость при работе с деревом, позволяет эффективно обрабатывать произвольную структуру и упрощает реализацию операций над деревом.

Могло быть использовано представление дерева в виде статического массива, где индексы элементов массива представляли бы узлы дерева. Например, для бинарного дерева левый потомок узла на позиции i размещался бы на позиции $2i + 1$, а правый — на позиции $2i + 2$. Однако такой подход требует заранее определённой структуры дерева (например, полного бинарного дерева) и объёма памяти, который выделяется под все возможные узлы. В случае деревьев с нерегулярной структурой или когда большая часть узлов остаётся пустой, это приводит к значительным издержкам в использовании памяти. Кроме того, статический массив не подходит для динамического роста дерева, который возникает при его построении по случайным правилам.

Использование машинных слов или массивов битов, эффективных в предыдущих лабораторных работах для представления множеств, также не подходит в данной задаче. Эти методы подходят для компактного хранения данных и быстрого выполнения операций, но требуют фиксированной

структуры данных. В случае дерева, где структура может быть произвольной, они теряют свою универсальность и усложняют реализацию базовых операций, таких как обход или вычисление высоты.

Выбранная структура с использованием указателей позволяет динамически выделять память только под существующие узлы, избегая затрат на неиспользуемые элементы. Она также обеспечивает удобство в реализации рекурсивных операций, таких как обход в глубину или вычисление высоты, которые естественным образом выражаются через указатели. Это делает данный подход наиболее подходящим для задач, связанных с динамическими деревьями.

Результаты прогона программы с генерацией случайного дерева

```
.....s.....
.....f.....r.....
.....c.....e.....j.....q.....
.....b.....d.....g.....i.....m.....p.....
.....a.....h.....k.....l.....o.....
.....n.....
DFS traversal: s f c b a e d r j g i h q m k l p o n
Height of the left subtree of the root: 4

Process finished with exit code 0
```

Рисунок 1 – Пример работы программы

Оценки временной сложности для каждой функции

Функция	Временная сложность
Создание дерева	$O(n)$
Обход дерева	$O(n)$
Вывод дерева	$O(n)$
Вычисление высоты	$O(n)$

Таблица 1 – Временная сложность функций

Создание дерева происходит за $O(n)$, где n – количество вершин в дереве. Обход также происходит за $O(n)$, так как каждый узел обрабатывается ровно один раз, добавляется в стек и удаляется из него. Вывод дерева происходит за $O(n)$, потому что дерево выводится рекурсивно. Для каждого узла обновляется экранный буфер, и функция вызывается для его потомков.

Для вычисления высоты левого поддерева мы проходимся по всем вершинам левого поддерева, поэтому сложность также $O(n)$.

Выводы

В рамках выполненной лабораторной работы были изучены и реализованы основные алгоритмы работы с двоичным деревом, включая создание дерева, обход в глубину, визуализацию и вычисление высоты левого поддерева. Программа была построена с использованием динамических структур данных, что обеспечило гибкость в работе с деревом произвольной структуры.

Использование указателей для представления узлов дерева позволило динамически создавать дерево и выполнять операции над его элементами с минимальными затратами памяти, выделяемой только для существующих узлов. Это подтвердило эффективность выбранной структуры данных для работы с нерегулярными деревьями.

Статические массивы и массивы битов, несмотря на свою компактность, оказались менее подходящими для текущей задачи из-за ограничений в гибкости и высокой зависимости от заранее известной структуры данных.

Программа была протестирована на деревьях с различной глубиной и показала корректные результаты.

Список использованной литературы

1. Колинко П. Г. Пользовательские структуры данных: Методические указания по дисциплине «Алгоритмы и структуры данных, часть 1». — СПб.: СПбГЭТУ «ЛЭТИ», 2024. — 64 с. (вып.2408).

2. Алгоритмы и структуры данных. Лекции на сайте <https://vec.etu.ru/moodle/course/view.php?id=13286>.

Приложение

Код программы

```
#include <iostream>
#include <cstdlib>
#include <ctime>

template<typename T>
class STACK {
private:
    T *stackArray;
    int topIndex;
    int capacity;

public:
    STACK(int maxCapacity = 100) : capacity(maxCapacity), topIndex(0) {
        stackArray = new T[capacity];
    }

    ~STACK() {
        delete[] stackArray;
    }

    bool empty() const {
        return topIndex == 0;
    }

    void push(const T &item) {
        if (topIndex >= capacity) {
            throw std::overflow_error("Stack overflow");
        }
        stackArray[topIndex++] = item;
    }

    T pop() {
        if (empty()) {
            throw std::underflow_error("Stack underflow");
        }
        return stackArray[--topIndex];
    }
};

class Node {
public:
    char label;
    Node *left;
    Node *right;

    Node() : label('\0'), left(nullptr), right(nullptr) {}

    ~Node() {
        delete left;
        delete right;
    }
};

class Tree {
public:
    Tree();

    ~Tree();
};
```

```

void MakeTree();

void OutTree();

void DFS();

int GetLeftSubtreeHeight();

private:
    Node *root;

    void MakeTree(Node *&node, int depth);

    void OutTree(Node *node, int x, int y, char screen[][120], int maxrow, int spacing);

    int GetHeight(Node *node);

    void LabelNodes(Node *node, char &currentLabel);

    Tree(const Tree &) = delete;

    Tree &operator=(const Tree &) = delete;

    Tree(Tree &&) = delete;

    Tree &operator=(Tree &&) = delete;
};

Tree::Tree() : root(nullptr) {}

Tree::~~Tree() {
    delete root;
}

void Tree::MakeTree() {
    MakeTree(root, 1);
    char currentLabel = 'a';
    LabelNodes(root, currentLabel);
}

void Tree::MakeTree(Node *&node, int depth) {
    if (depth > 6) {
        return;
    }

    if (depth == 1 || depth <= rand() % 6 + 1) {
        node = new Node();
        MakeTree(node->left, depth + 1);
        MakeTree(node->right, depth + 1);
    } else {
        node = nullptr;
    }
}

void Tree::LabelNodes(Node *node, char &currentLabel) {
    if (node == nullptr) {
        return;
    }

    LabelNodes(node->left, currentLabel);
    LabelNodes(node->right, currentLabel);
    node->label = currentLabel++;
    if (currentLabel > 'z')

```

```

        currentLabel = 'a';
    }

void Tree::OutTree() {
    if (root == nullptr) {
        std::cout << "Tree is empty!" << std::endl;
        return;
    }

    const int maxrow = 6;
    const int maxcol = 120;
    char screen[maxrow][maxcol];

    for (int i = 0; i < maxrow; ++i) {
        for (int j = 0; j < maxcol; ++j) {
            screen[i][j] = '.';
        }
        screen[i][maxcol - 1] = '\0';
    }

    OutTree(root, maxcol / 2, 0, screen, maxrow, maxcol / 4);

    for (int i = 0; i < maxrow; ++i) {
        std::cout << screen[i] << std::endl;
    }
}

void Tree::OutTree(Node *node, int x, int y, char screen[][120], int maxrow, int spacing) {
    if (node == nullptr || y >= maxrow || x < 0 || x >= 120) {
        return;
    }

    screen[y][x] = node->label;
    if (node->left) {
        OutTree(node->left, x - spacing, y + 1, screen, maxrow, std::max(spacing / 2, 2));
    }
    if (node->right) {
        OutTree(node->right, x + spacing, y + 1, screen, maxrow, std::max(spacing / 2, 2));
    }
}

void Tree::DFS() {
    if (root == nullptr) {
        std::cout << "Tree is empty!" << std::endl;
        return;
    }

    STACK<Node *> stack(100);
    stack.push(root);
    std::cout << "DFS traversal: ";
    while (!stack.empty()) {
        Node *node = stack.pop();
        std::cout << node->label << ' ';
        if (node->right)
            stack.push(node->right);
        if (node->left)
            stack.push(node->left);
    }
    std::cout << std::endl;
}

int Tree::GetLeftSubtreeHeight() {
    if (root == nullptr || root->left == nullptr) {

```



```

        return 0;
    }

    return GetHeight(root->left);
}

int Tree::GetHeight(Node *node) {
    if (node == nullptr) {
        return 0;
    }

    return 1 + std::max(GetHeight(node->left), GetHeight(node->right));
}

int main() {
    srand(static_cast<unsigned int>(time(nullptr)));
    Tree tree;

    tree.MakeTree();
    tree.OutTree();
    tree.DFS();
    std::cout << "Height of the left subtree of the root: " << tree.GetLeftSubtreeHeight() <<
std::endl;
    return 0;
}

```